

Handling Image

Wednesday, April 22, 2026 11:09 PM

Loading images to PyTorch

```
from torchvision.datasets import ImageFolder
from torchvision import transforms

train_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize((128, 128)),
])

dataset_train = ImageFolder(
    "data/clouds_train",
    transform=train_transforms,
)
```

- Define transformations:
 - Parse to tensor
 - Resize to 128×128
- Create dataset passing:
 - Path to data
 - Predefined transformations

Displaying images

```
dataloader_train = DataLoader(
    dataset_train,
    shuffle=True,
    batch_size=1,
)

image, label = next(iter(dataloader_train))
print(image.shape)
```

```
torch.Size([1, 3, 128, 128])
```

```
image = image.squeeze().permute(1, 2, 0)
print(image.shape)
```

```
torch.Size([128, 128, 3])
```

```
import matplotlib.pyplot as plt
plt.imshow(image)
plt.show()
```



◆ Main Idea

Image → Transform → Dataset → DataLoader → Model

👉 Convert raw images into tensors → feed into neural network

◆ Loading Images (Core Workflow)

1. Define Transformations

```
from torchvision import transforms
train_transforms = transforms.Compose([
    transforms.ToTensor(),
    transforms.Resize((128, 128)),
])
```

👉 What happens:

- Convert image → tensor
- Resize all images to same size

2. Create Dataset

```
from torchvision.datasets import ImageFolder
dataset_train = ImageFolder(
    "data/clouds_train",
    transform=train_transforms
)
```

👉 Folder structure = labels
(each subfolder = class)

◆ DataLoader (Batching)

```
from torch.utils.data import DataLoader
dataloader_train = DataLoader(
    dataset_train,
    batch_size=1,
    shuffle=True
)
```

👉 Handles:

- batching
- shuffling
- iteration

◆ Image Shape (Very Important)

[batch, channels, height, width]
→ [1, 3, 128, 128]

- 1 = batch size
- 3 = RGB channels
- 128×128 = image size

◆ Displaying Images

Convert shape for visualization:

```
image = image.squeeze().permute(1, 2, 0)
```

👉 Why:

◆ What is an Image

- Image = grid of **pixels**
- Each pixel = numeric values

Types:

- **Grayscale** → 1 value (0–255)
- **RGB (color)** → 3 values (R, G, B)

e?

B)

- Remove batch dim → [3,128,128]
- Reorder → [128,128,3] (for matplotlib)

```
plt.imshow(image)
plt.show()
```

◆ Data Augmentation

Example:

```
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(45),
    transforms.ToTensor(),
    transforms.Resize((128, 128)),
])
```

◆ Why Data Augmentation Matters

Benefits ✓

- Increase dataset size artificially
- Improve model robustness
- Reduce overfitting

👉 Model learns:
"ignore small variations"

◆ Key Concepts to Remember

- Images → tensors (required for PyTorch)
- Shape format = **[B, C, H, W]**
- Use ImageFolder for structured datasets
- Augmentation = random transformations

Exercise

Instructions

100 XP

- Add two more transformations to `train_transforms` to perform a random horizontal flip and then a rotation by a random angle between 0 and 45 degrees.
- Reshape the `image` tensor from the DataLoader to make it suitable for display.
- Display the image.

 Take Hint (-30 XP)

There was an error in your code

The error comes from the following part of the code:

```
plt.show()
```

What is the issue?

The problem here is that `plt.show()` is not valid Python syntax. It looks like a placeholder, possibly left as a reminder to add a line of code. In Python, every statement must be valid code, and `plt.show()` is not defined anywhere, so Python raises a `NameError`. This error means that Python tried to find a variable or function named `plt.show()`, but it doesn't exist.

How do you fix it?

To fix this, simply remove the placeholder line. You can display the image directly with `plt.imshow(image)` after your previous code. There is no need for an extra line or placeholder before calling `plt.imshow(image)`.

Here's the corrected code segment:

```
plt.imshow(image)
```

This should resolve the error and display your image as intended.

Was the AI Assistant helpful?

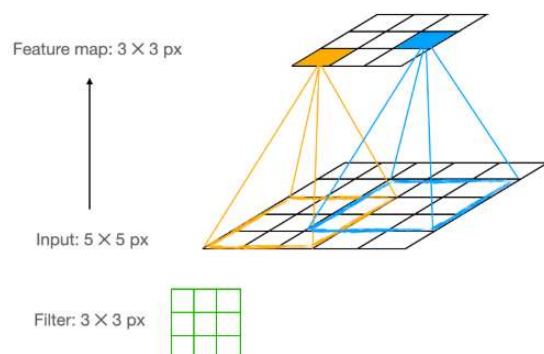
☒ Yes ☐ No

script.py

```
1 train_transforms = transforms.Compose(
2     # Add horizontal flip and rotation
3     transforms.RandomHorizontalFlip(),
4     transforms.RandomRotation(45),
5     transforms.ToTensor(),
6     transforms.Resize((128, 128))
7 )
8
9 dataset_train = ImageFolder(
10     "clouds_train",
11     transform=train_transforms,
12 )
13
14 dataloader_train = DataLoader(
15     dataset_train, shuffle=True, batch_size=16
16 )
17
18 image, label = next(iter(dataloader_train))
19 # Reshape the image tensor
20 image = image.squeeze().permute(1, 2, 0)
21 # Display the image
22 plt.imshow(image)
23 plt.show()
```

Convolutional Neural Networks (CNN)

Convolutional layer



- Slide filter(s) of parameters over the input
- At each position, perform convolution
- Resulting feature map:
 - Preserves spatial patterns from input
 - Uses fewer parameters than linear layer
- One filter = one feature map
- Apply activations to feature maps
- All feature maps combined form the output
- `nn.Conv2d(3, 32, kernel_size=3)`

Daily XP 250 EN

Light Mode

Plots ↗

```

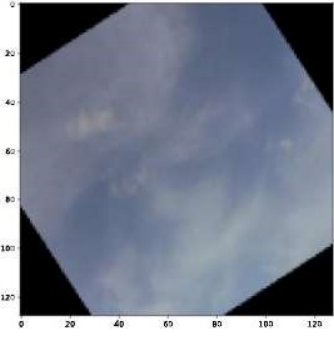
pose([
ation
ip(),
,
,
,

atch_size=1

er_train))

, 2, 0)

```



← Previous Plot 2/2 Next Plot →

Run Code Submit Answer

1. Convolutional Layer

- Slides **filters (kernels)** over image
- Performs **dot product + sum**

👉 Output = **Feature Map**

Key idea:

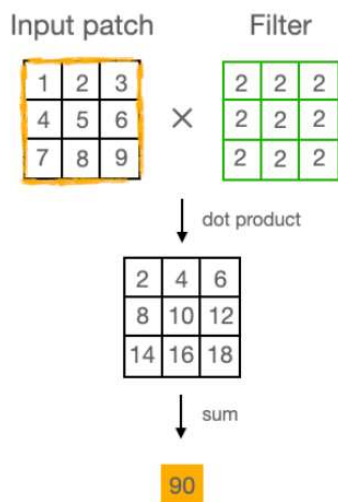
- 1 filter → 1 feature map
- Learns patterns like edges, textures

`nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)`

✓ Preserves spatial structure

✓ Uses fewer parameters (page 16)

Convolution



1. Compute dot product of input patch and filter
 - Top-left field: $2 \times 1 = 2$

Zero-padding

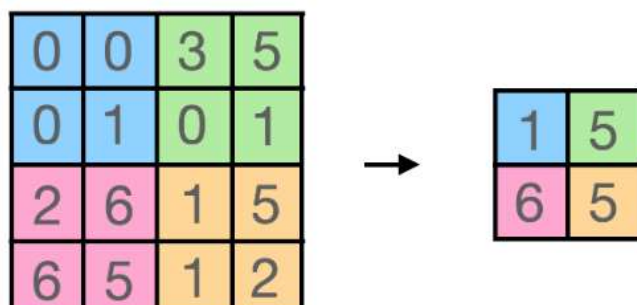
0	0	0	0	0	0
0					0
0					0
0					0
0					0
0	0	0	0	0	0

- Add a frames of zeros to convolutional layer's input

```
nn.Conv2d(  
    3, 32, kernel_size=3, padding=1  
)
```

- Maintains spatial dimensions of the input and output tensors
- Ensures border pixels are treated equally to others

Max Pooling



- Slide non-overlapping window over input
- At each position, retain only the maximum value
- Used after convolutional layers to reduce spatial dimensions
- `nn.MaxPool2d(kernel_size=2)`

Convolutional Neural Network

```
class Net(nn.Module):
```

- `feature_extractor : (convolution.`

2. Convolution Operation

- Take small patch $\times$ filter
- Multiply element-wise
- Sum result

👉 This extracts features

3. Zero Padding

- Add zeros around borders

```
nn.Conv2d(3, 32, kernel_size=3, padding=1)
```

- ✓ Keeps output size same as input
- ✓ Preserves edge information (page 18)

4. Activation Function

- Adds non-linearity

Example:

```
nn.ELU()
```

- ✓ Enables learning complex patterns

5. Max Pooling

- Downsamples feature map
- Keeps **maximum value in window**

```
nn.MaxPool2d(kernel_size=2)
```

- ✓ Reduces size (H, W $\downarrow$)
- ✓ Keeps important features (page 19)

6. Flatten + Linear Layer

- Convert feature maps $\rightarrow$ vector
- Final classification

```
nn.Flatten()
```


Convolutional Neural Network

```
class Net(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.feature_extractor = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.ELU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ELU(),
            nn.MaxPool2d(kernel_size=2),
            nn.Flatten(),
        )
        self.classifier = nn.Linear(64*16*16, num_classes)

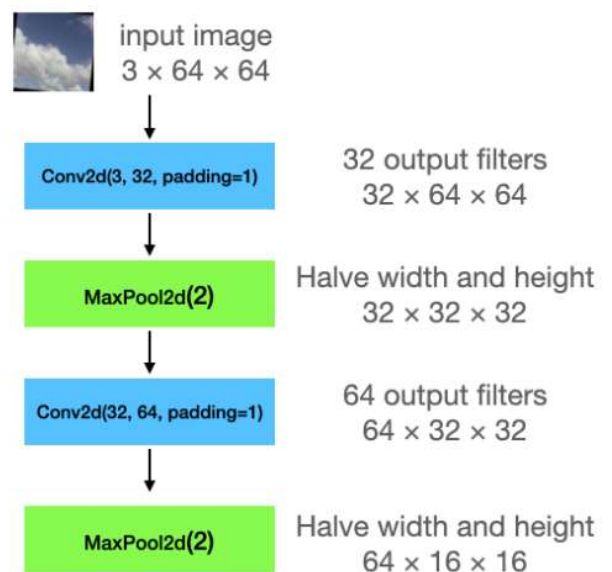
    def forward(self, x):
        x = self.feature_extractor(x)
        x = self.classifier(x)
        return x
```

- `feature_extractor` : (convolution, activation, pooling), repeated twice flattened
- `classifier` : single linear layer
- `forward()` : pass input image through feature extractor and classifier

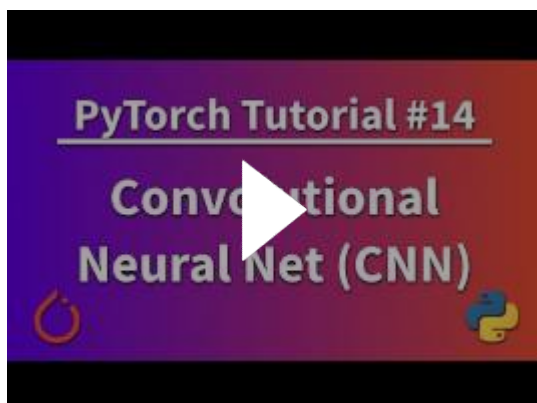
Feature extractor output size

```
self.feature_extractor = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, padding=1),
    nn.ELU(),
    nn.MaxPool2d(kernel_size=2),
    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.ELU(),
    nn.MaxPool2d(kernel_size=2),
    nn.Flatten(),
)

self.classifier = nn.Linear(64*16*16, num_classes)
```



PyTorch Tutorial 14 - Convolutional Neural Network (CNN)



- **Building a CNN**
composed of convolutional layers
vector of features
these features.
`nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, stride=1, padding=1)`
- This layer takes an input of size 1x1x1x1 and preserves spatial dimensions.

From <<https://camnetworks?ex=8>>

- Final classification

```
nn.Flatten()
nn.Linear(64*16*16, num_classes)
```

◆ CNN Architecture Pattern (IMPORTANT)

Typical flow:

```
Conv → Activation → Pool
→ Conv → Activation → Pool
→ Flatten → Linear
```

◆ Feature Map Size Flow (VERY IMPORTANT)

Example from slides (pages 21–25):

Input:

$3 \times 64 \times 64$

After layers:

Conv → $32 \times 64 \times 64$

Pool → $32 \times 32 \times 32$

Conv → $64 \times 32 \times 32$

Pool → $64 \times 16 \times 16$

Flatten → $64 \times 16 \times 16$

👉 Pooling halves width & height each time

CNN in PyTorch: You practiced defining a CNN with two main parts: a feature extractor (convolutional and max pooling layers), and a classifier. The feature extractor learns a representation from the input image, while the classifier predicts the image's class based on this representation.

For example, defining a convolutional layer in PyTorch looks like this:

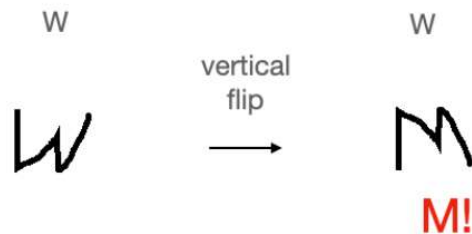
```
nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
```

This layer takes an input with 3 channels (RGB) and applies 32 filters of size 3x3, using padding to maintain spatial dimensions.

https://pytorch.org/tutorials/beginner/convolution_2d_tutorial.html

Augmentation:

What should not be augmented



- Augmentations can impact the label
- Whether this is confusing depends on the task
- Always choose augmentations with the data and task in mind!

Augmentations for cloud classification



- **Random rotation:** expose model to different angles of cloud formations
- **Horizontal flip:** simulate different viewpoints of the sky
- **Auto contrast adjustment:** simulate different lighting conditions

```
train_transforms = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(45),
    transforms.RandomAutocontrast(),
    transforms.ToTensor(),
    transforms.Resize((128, 128))
])
```

Cross-Entropy loss

- Binary classification: binary cross-entropy (BCE) loss
- Multi-class classification: cross-entropy loss
- `criterion = nn.CrossEntropyLoss()`

Image classifier training loop

```
net = Net(num_classes=7)
```

◆ What is Cross-Entropy Loss?

A loss function used for **classification problems**:

- Measures difference between:
 - **Predicted probabilities**
 - **True labels**

👉 Goal: minimize this difference

◆ When to Use

Binary Classification

- Use **Binary Cross-Entropy (BCE)**

Multi-Class Classification (CNN case)

- Use **CrossEntropyLoss**

```
criterion = nn.CrossEntropyLoss()
```

✓ Used when:

- Output = multiple classes (e.g., 7 cloud types)

◆ How It Works (Intuition)

1. Model outputs **logits** (raw scores)
2. Internally applies:
 - **Softmax** → **probabilities**
3. Compares with true label
4. Penalizes wrong predictions

👉 Key idea:

- Correct class → high probability → low loss
- Wrong class → low probability → high loss

◆ Important Behavior

- You **DO NOT** apply **Softmax** manually
- **CrossEntropyLoss** already includes it

✗ Wrong:

```
softmax(outputs)
```

```
loss = CrossEntropyLoss(...)
```

✓ Correct:

```
loss = criterion(outputs, labels)
```

◆ Key Pattern to Remember

Model output (logits)

→ **CrossEntropyLoss**

→ computes error

◆ Important Terms

different

Step-by-Step Breakdown

image classifier training loop

```
net = Net(num_classes=7)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(net.parameters(), lr=0.001)

for epoch in range(10):
    for images, labels in dataloader_train:
        optimizer.zero_grad()
        outputs = net(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

5. Update Weights

optimizer.step()

- Adjust weights using gradients

Step

1. optim

- Clear
- Preve

2. Forw

- Input

3. Com

loss = crite

- Com

4. Backp

loss.backward

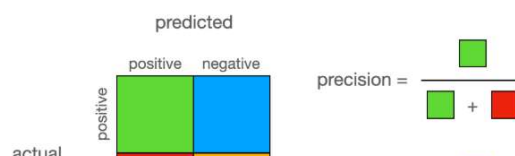
- Comp

Evaluating Image Classifier

Precision & Recall: binary classification

In **binary** classification:

- Precision:** Fraction of correct positive predictions
- Recall:** Fraction of all positive examples correctly predicted



Step-by-Step Breakdown

optimizer.zero_grad()

• Resets previous gradients
• Prevents gradients accumulation

Forward Pass

Input image → CNN → predictions

Compute Loss

• criterion(outputs, labels)
• Compares prediction vs true label

Backward Propagation

• optimizer.backward()
• Compute gradients ($\partial \text{Loss} / \partial \text{Weights}$)

→ computes error



Important Terms

- **Logits** → raw model output
- **Softmax** → converts to probabilities
- **Target label** → class index (not one-hot)



Multi-class Metric Averaging

Micro

- Global calculation across all samples
- Dominated by **majority classes**

Micro: $\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C \text{count}_{ij}$

Averaging — Quick Memo

(TP, FP, FN summed first)

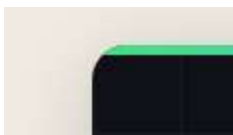
Averaging Multi-Class Metrics

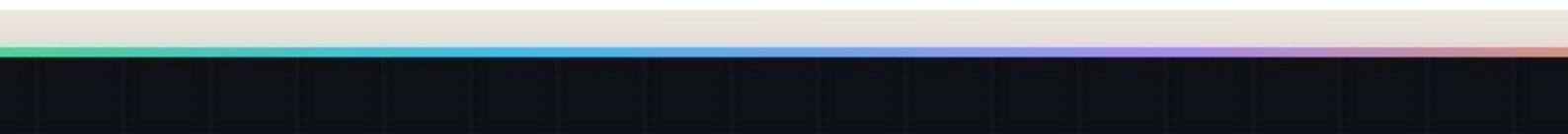
When you have **multiple classes** (like 7 cloud types), metrics are calculated for each class. The `average=` option tells PyTorch **how to combine scores**.

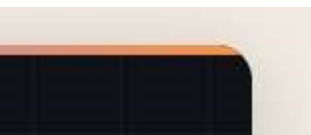
1. `average=None`

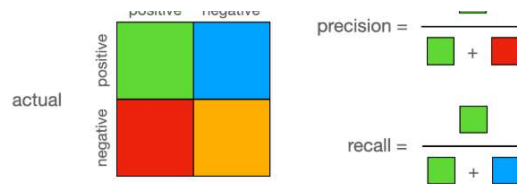
(Simple language)

Metrics like **Recall**, **Precision**, or **F1-score** can be calculated in different ways.
from all classes.









Precision & Recall: multi-class classification

In **multi-class classification**: separate precision and recall for each class

- **Precision**: Fraction of cumulus-predictions that were correct
- **Recall**: Fraction of all cumulus examples correctly predicted



Averaging

- With 7
- We can
 - Micro
 - Macro
 - Weighted

Averaging multi-class metrics

```
from torchmetrics import Recall

recall_per_class = Recall(task="multiclass", num_classes=7, average=None)
recall_micro = Recall(task="multiclass", num_classes=7, average="micro")
recall_macro = Recall(task="multiclass", num_classes=7, average="macro")
recall_weighted = Recall(task="multiclass", num_classes=7, average="weighted")
```

When to use each:

- Micro: Imbalanced datasets
- Macro: Care about performance on small classes
- Weighted: Consider errors in larger classes as more important

Evaluation loop

```
from torchmetrics import Precision, Recall

metric_precision = Precision(
```

- Import and define metrics

Averaging multi-class metrics

For 7 classes, we have 7 precision and 7 recall scores

We can analyze them per-class, or aggregate:

Micro average: global calculation

Macro average: mean of per-class metrics

Weighted average: weighted mean of per-class metrics

- Global calculation across all samples
- Dominated by **majority classes**
- ☒ Best for **imbalanced datasets**
- 📌 Reflects overall model performance

Macro

- Compute metric **per class** → then average
- Treats all classes **equally**
- 📌 Ignores class imbalance
- 📌 Good when **small classes matter**

Weighted

- Compute per class → average using class weights
- Balance between micro & macro
- 📌 Larger classes have **more influence**

🔄 Quick Comparison

Type	Handles Imbalance	Focus
Micro	☒ Yes	Overall performance
Macro	☒ No	Equal class weight
Weighted	☒ Partial	Proportional to class size

⚡ Key Insight (from your slides)

- Micro → imbalanced datasets
- Macro → care about **small classes**
- Weighted → emphasize **larger classes**

🚀 One-line Memory Trick

- **Micro** = global
- **Macro** = equal
- **Weighted** = proportional

From <<https://chatgpt.com/g/g-p-67f5e6c8a53c8191b26f23746a9a>>

```
precision)
recall)
weighted")
```

the precision and recall

(TP, FP, FN summed first)

nce

erage

equally

class size (support) as weights

ce, but all classes included

formance

importance

al importance

de)

s more

<https://la64a9f17cbd1a875-ai/c/69f2c2f4-6a6c-83ea-9105->

the average= option tells Python how to combine scores

1. average=None

Meaning

Return **one score per class**

Recall(task="multiclass", num_classes=7, average=None)

You get:

[class0_score, class1_score, class2_score, ...]

Simple idea

You see **individual performance** for each class separately.

Example

Class	Recall
Cat	0.95
Dog	0.88
Bird	0.60

You can clearly detect:

- which class performs well
- which class struggles

Use when

- You want detailed analysis
- You want to inspect weak classes
- Debugging model performance

2. average="micro"

Meaning

Combine **all predictions together first**, then compute one

Recall(task="multiclass", num_classes=7, average="micro")

Simple idea

Treat every sample equally.

Big classes naturally affect the result more because they h

Why good for imbalanced datasets

Example dataset:

Class	Samples
Cats	900
Dogs	90
Birds	10

Micro averaging mainly reflects performance on:

- Cats (large class)

because most samples belong there.

Mental model

Count ALL correct predictions together

÷

Count ALL total samples together

Use when

- Dataset is highly imbalanced

from all classes.

overall score.
)

ave more samples.

?



Averaging Multi-Class Metrics

strategies to summarize recall across 7 classes — and when to use each

`average=None`

Per-class scores

Returns 7 recall values. No combining. Raw transparency per class.

`average="micro"`

Global pooling

Pools all TPs & FNs first. Big classes dominate the result.

`average="macro"`

Equal weight

Simple mean of all class scores. Every class counts equally.

`average="weighted"`

Size-weighted mean

Weighted by class frequency. Larger classes influence more.

from torchmetrics

1 OF 4

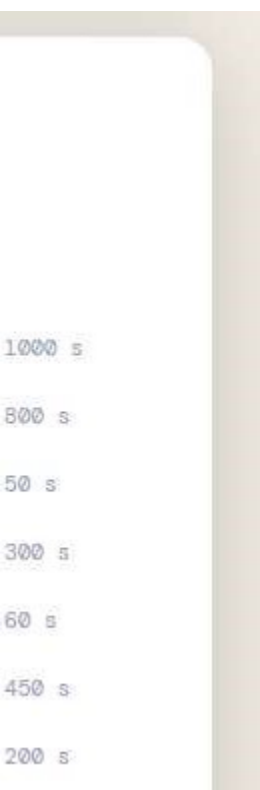
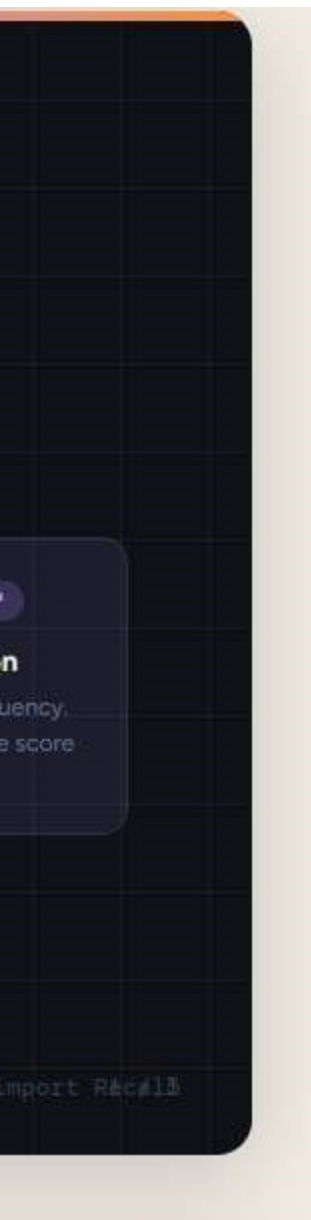
Per-Class Recall

`average=None`

one individual score for each class. Averaging happens — you see every recall separately.

RECALL PER CLASS (7 VALUES RETURNED)





```
metric_precision = Precision(
    task="multiclass", num_classes=7, average="macro"
)
metric_recall = Recall(
    task="multiclass", num_classes=7, average="macro"
)
net.eval()
with torch.no_grad():
    for images, labels in dataloader_test:
        outputs = net(images)
        _, preds = torch.max(outputs, 1)
        metric_precision(preds, labels)
        metric_recall(preds, labels)
precision = metric_precision.compute()
recall = metric_recall.compute()
```

metrics

- Iterate over test
- For each test batch, take most likely predicted class and compute precision and recall functions along with
- Compute the metrics

```
print(f"Precision: {precision}")
print(f"Recall: {recall}")
```

```
Precision: 0.72840106
Recall: 0.76303869485
```

Analyzing performance per class

```
metric_recall = Recall(
    task="multiclass", num_classes=7, average=None
)
net.eval()
with torch.no_grad():
    for images, labels in dataloader_test:
        outputs = net(images)
        _, preds = torch.max(outputs, 1)
        metric_recall(preds, labels)
recall = metric_recall.compute()
```

```
print(recall)
```

```
tensor([0.6364, 1.0000, 0.9091, 0.7917,
        0.5049, 0.9500, 0.5493],
       dtype=torch.float32)
```

- Compute metric with `average=None`
- This gives one score per class
- Dataset's `.class_to_idx` maps class names to indices

```
dataset_test.class_to_idx
```

```
{'cirriform clouds': 0,
 'clear sky': 1,
 'cumulonimbus clouds': 2,
 'cumulus clouds': 3,
 'high cumuliiform clouds': 4,
 'stratiform clouds': 5,
 'stratocumulus clouds': 6}
```

Analyzing performance per class

```
{
    k: recall[v].item()
    for k, v
    in dataset_test.class_to_idx.items()
}
```

- `k` = class name
- `v` = class index
- `recall[v]` = recall for class `v`
`tensor(0.6364)`

examples with no gradient
atch, get model outputs,
class, and pass it to metric
with the labels

etrics

```
recision}")  
ll}")
```

```
48727417  
8551
```

```
age=None
```

```
ss
```

attribute maps

me, e.g. cirriform clouds

ex, e.g. 0

```
4, dtype=torch.float32)
```

Use when

- Dataset is highly imbalanced
- You care about overall system accuracy
- Every sample matters equally

3. average="macro"

Meaning

Calculate metric for each class separately, then average them.
`Recall(task="multiclass", num_classes=7, average="macro")`

Simple idea

Every class gets the **same importance**, even tiny classes.

Example

Class	Recall
Cats	0.95
Dogs	0.88
Birds	0.20

Macro recall:

$(0.95 + 0.88 + 0.20) / 3$

Even though Birds has few samples, it affects the score equally.

Why useful?

It exposes poor performance on small classes.

Without macro averaging:

- model may ignore small classes
- overall score may still look good

Use when

- Small classes are important
- You want fairness across classes
- You do not want large classes dominating results

4. average="weighted"

Meaning

Like macro averaging, BUT each class gets weighted by its size.
`Recall(task="multiclass", num_classes=7, average="weighted")`

Simple idea

Large classes influence the result more.

But unlike micro:

- it still computes metric per class first

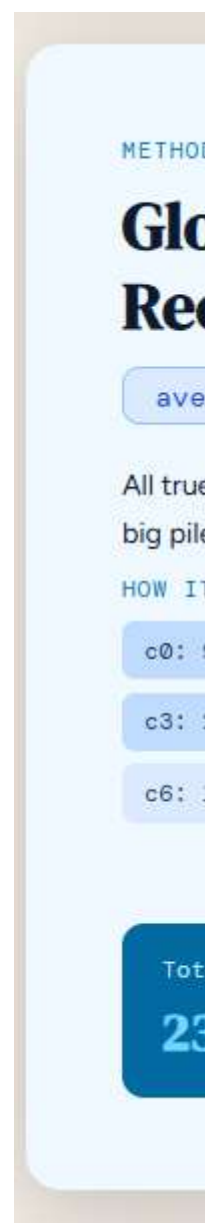
Example

Class	Recall	Samples
Cats	0.95	900
Dogs	0.88	90

them equally.
")

ually.

dataset size.
ted")



recall separately.

N

t to debug individual classes or spot
es are underperforming.

METHOD 1 OF 4

class 6

0.55

```
OUTPUT + tensor([
  0.90, 0.75, 0.60, 0.85, 0.40, 0.95, 0.55 ])
```

D 2 OF 4

Global Pooling call

range="micro"

the positives (TP) and false negatives (FN) from every class are pooled into one
e before computing recall.

T POOLS

900 TP / 1000

c1: 600 TP / 800

c2: 30 TP / 50

255 TP / 300

c4: 24 TP / 60

c5: 427 TP / 450

110 TP / 200

↓ add all together ↓

al TP

346

Total samples

2860

Micro Recall

0.82

KEY INSIGHTS



Big classes dominate

Class 0 (1000 samples) has 35% influence than class 2 (50 samples). recall pulls the score up.



Hides rare class failures

Class 4 has only 0.40 recall, but makes it nearly invisible in the micro recall.



Use when: imbalanced data

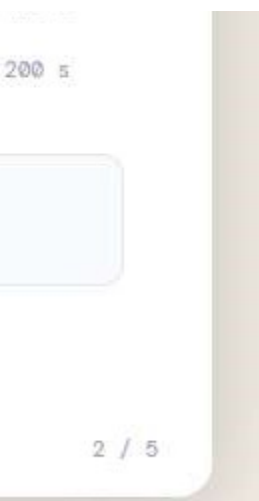
Great when you care about overall predictions and accept that large classes matter more.

METHOD 3 OF 4

Equal Weight

EACH CLASS = 1/7 WEIGHT





```
}
```

```
{'cirriform clouds': 0.6363636255264282,  
 'clear sky': 1.0,  
 'cumulonimbus clouds': 0.9090909361839294,  
 'cumulus clouds': 0.7916666865348816,  
 'high cumuliiform clouds': 0.5048543810844421,  
 'stratiform clouds': 0.949999988079071,  
 'stratocumulus clouds': 0.5492957830429077}
```

```
recall[v]  
tensor(0.636
```

```
• recall[v].it
```

Exercise

Analyzing metrics per class

While aggregated metrics are useful indicators of the model's performance, it is often informative to look at the metrics per class. This could reveal classes for which the model underperforms.

In this exercise, you will run the evaluation loop again to get our cloud classifier's precision, but this time per-class. Then, you will map these score to the class names to interpret them. As usual, `Precision` has already been imported for you. Good luck!

Instructions

100 XP

- Define a precision metric appropriate for per-class results.
- Calculate the precision per class by finishing the dict comprehension, iterating over the `.items()` of the `.class_to_idx` attribute of `dataset_test`.



Take Hint (-30 XP)

There was an error in your code

The error comes from the following part of the code:

```
precision_per_class = {  
    k: precision[v].item()  
    for k, v  
    in dataset_test.class_to_idx.items()  
}
```



What is the issue?

The problem arises because you defined your precision metric with `average="macro"`. When you use `"macro"` averaging in `torchmetrics.Precision`, the result is a single scalar value representing the average precision across all classes, not a tensor with one value per

```
4, dtype=torch.float32)
```

```
em() = 0.6364
```

script.py

Light M

```
1 # Define precision metric
2 metric_precision = Precision[
3     task="multiclass", num_classes=7, average=None
4 ]
5
6 net.eval()
7 with torch.no_grad():
8     for images, labels in dataloader_test:
9         outputs = net(images)
10        _, preds = torch.max(outputs, 1)
11        metric_precision(preds, labels)
12 precision = metric_precision.compute()
13
14 # Get precision per class
15 precision_per_class = {
16     k: precision[v].item()
17     for k, v
18     in dataset_test.class_to_idx.items()
19 }
20 print(precision_per_class)
```



Run Code

Submit Answer

IPython Shell

Slides

```
# Get precision per class
precision_per_class = {
    k: precision[v].item()
    for k, v
    in dataset_test.class_to_idx.items()
}
print(precision_per_class)
```

```
{'cirriform clouds': 0.699999988079071, 'clear sky': 0.9384615421295166, 'cumulonimbus clouds': 0.80000001192}
```

Cats	0.95	900
Dogs	0.88	90
Birds	0.20	10

Weighted average is dominated by Cats because Cats has

Difference: Micro vs Weighted

They sound similar but work differently.

Micro

Combine ALL predictions first
→ compute one metric

Weighted

Compute metric per class first
→ average using class sizes

Easy Memory Trick

Type	Think
None	"Show every class separately"
Micro	"Every sample equal"
Macro	"Every class equal"
Weighted	"Every class counted by size"

Visual Intuition

Dataset:

Cats = 900 samples

Dogs = 90 samples

Birds = 10 samples

Micro

Large classes dominate naturally

Macro

All 3 classes treated equally

Weighted

Balanced between both ideas

Typical Recommendation

Situation	Best Choice
Debugging classes	None
Imbalanced dataset	micro
Care about rare classes	macro
Want balanced realistic overall score	weighted

Code Pattern

```
from torchmetrics import Recall
# One score per class
```

many samples.

Equal Weight Recall

average="macro"

Calculates recall per class, then takes a plain average. Every class has identical influence — even a class with only 50 samples counts the same as one with 1000.

$$\begin{aligned} \text{Recall_macro} &= (R_0 + R_1 + R_2 + R_3 + R_4 + R_5 + R_6) \div 7 \\ &= (0.90 + 0.75 + 0.60 + 0.85 + 0.40 + 0.95 + 0.55) \div 7 \\ &= 5.00 \div 7 = 0.714 \end{aligned}$$

EACH CLASS = 1/7 WEIGHT

c0	c1	c2	c3	c4	c5
0.90	0.75	0.60	0.85	0.40	0.95
1/7	1/7	1/7	1/7	1/7	1/7

← poor classes (c2, c4) pull the average significantly →

0.714

Lower than micro (0.82) because c2 and c4 are fully counted despite tiny sample sizes.

Size-Weighted Recall

average="weighted"

Like macro, but each class recall is multiplied by the class's proportion of the total data before averaging.

macro result

0.714

all classes = equal weight

weighted result

0.834

big classes = more weight

WEIGHT = PROPORTION OF TOTAL SAMPLES (28)

class 0 0.90 recall

class 1 0.75 recall

class 2

class 3 0.85

class 4

class 5 0.95

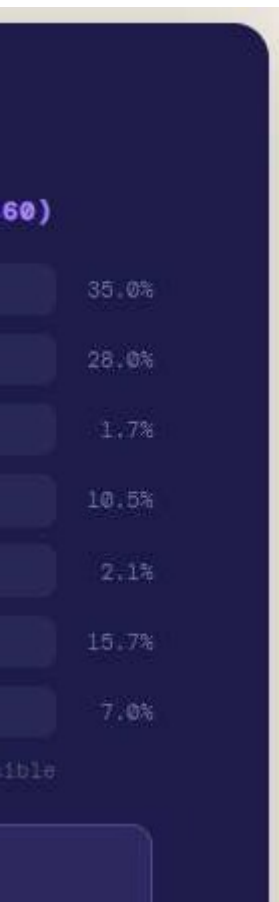
class 6 0.55

classes 2 & 4 have ~2% weight — nearly invisible

0.834

Much closer to micro (0.82).

c0 & c1 (63% of data) dominate



Code Pattern

```
from torchmetrics import Recall
# One score per class
recall_per_class = Recall(
    task="multiclass",
    num_classes=7,
    average=None
)
# Overall score (all samples together)
recall_micro = Recall(
    task="multiclass",
    num_classes=7,
    average="micro"
)
# Equal importance to all classes
recall_macro = Recall(
    task="multiclass",
    num_classes=7,
    average="macro"
)
# Larger classes weighted more
recall_weighted = Recall(
    task="multiclass",
    num_classes=7,
    average="weighted"
)
```

Quick Summary

None → one metric per class
Micro → all samples treated equally
Macro → all classes treated equally
Weighted → classes weighted by dataset size

From <<https://chatgpt.com/g/g-p-67f5e6c8a53c8191a64a9f17cbd1a87>>

[75-ai/c/6a0098ee-e238-83ea-8e90-0ad886d950e0>](#)

